PROJECT DELIVERY & QUALITY ASSURANCE REPORT

Northstar — Product Analytics Dashboard

High-fidelity, enterprise-grade frontend prototype — a complete SaaS product-analytics workbench delivered as a zero-dependency web app.

PROJECT	VERSION
Northstar | Analytics	**1.0**
PREPARED	STACK
07 Nov 2026	**HTML / CSS / Vanilla JS**

01 Executive Summary

A high-fidelity, enterprise-grade frontend prototype delivered as a zero-dependency single-page web app — written in pure native HTML, CSS and Vanilla JavaScript, with no framework, no build step and no external network request.

The product models a complete analytics workbench: an overview of KPIs, hand-built SVG charts, a users / cohorts area, a billing & invoices view with CSV export, a keyboard-first command palette (⌘K), and a client-side authentication screen. Unlike low-end dashboards assembled from a charting library and a UI kit, the data here is **real client-side state and logic** — a virtualized table mounts only the visible rows of a 5,000-row dataset, charts are computed from data, and a network-failure simulator lets you exercise the error / retry path on demand.

Under the hood sits a custom **NS infrastructure layer** (`app.js`, exposed as `window.NS`) that provides DOM helpers, a hash router with a route guard, skeleton loading states, a recoverable error + retry component, a windowed virtual-list engine, and debounce / throttle timing utilities. The same UI runs unchanged against mock data today and a live REST backend tomorrow — only the data source flag changes.

02 Core Features

Four capabilities distinguish this build from a template: each is implemented as deterministic logic, not decorative markup.

- **Command palette (⌘K / Ctrl-K)** — a global, keyboard-first surface for navigation and actions across every view. Opening, fuzzy searching and executing a command is real behaviour, not a hidden menu; focus is trapped, Escape closes, and the trigger is restored.
- **Hand-built SVG chart engine** — line, area, bar and donut visualizations are rendered from data with no charting library. Metrics carry deterministic deltas and sparkline companions; the visuals scale with the viewport and never overflow.
- **Virtualized data table** — the users view renders a 5,000-row dataset through a windowed engine (`VirtualTable.mount`): only the visible rows (plus a small overscan) are mounted in the DOM, and scroll repaints are coalesced with a throttle. This is genuine large-dataset rendering, not a static screenshot.
- **Client-side auth & route guard** — a polished login screen gates the app; deep-linking to a protected view while logged out is intercepted by `guardRoute` and bounced back to sign-in. Logout / login transitions are explicit and stateful.

03 UI / UX Design Specification

The interface is governed by a single token set declared as CSS custom properties — colour, type and spacing never drift, because nothing is hard-coded.

TOKEN	VALUE	ROLE
`--bg`	#0B0F19 · Abyss	Page background (dark theme)
`--bg-2`	#0E1422 · Abyss-2	Elevated surface, page bands
`--bg-3`	#111827 · Panel	Card / panel base
`--surface`	rgba(255,255,255,.035 / .055 / .085)	Glass cards (3 elevation levels)
`--txt`	#EAF0FA · Snow	Primary text & headings
`--txt-2`	#98A6BE · Slate	Secondary text
`--txt-3`	#64748B · Muted	Captions, metadata
`--line`	rgba(255,255,255,.075 / .13 / .20)	Hairlines & borders (3 levels)
`--blue`	#3B82F6 · Blue	Primary accent, CTAs, focus
`--cyan`	#22D3EE · Cyan	Gradient partner / 2nd accent
`--green`	#34D399 · Emerald	Positive / up metrics
`--red`	#F87171 · Red	Negative / down, alerts
`--amber`	#FBBF24 · Amber	Warning state
`--violet / --pink`	#A78BFA · #F472B6	Accent theme variants

SCALE / SPEC	VALUE	NOTE
Type	13.5/1.55 base · page-h h1 23/730 · -0.035em tracking	Body → Display, system-ui
Radius	16 · 10 · 7	Cards / inputs / chips
Elevation	0 18px 50px rgba(0,0,0,.34) + backdrop blur(20px)	Soft card shadow + glass
Theme	dark-first · color-scheme:dark · [data-accent]	Accent switch, no code change
Font	Native system-ui stack	No web-font download, instant render

04 Architecture & Code Standards

`app.js` exposes a single `window.NS` namespace — the one piece of shared infrastructure. It is framework-free, fully commented (M1–M5), and exposes a small, intentional API.

```
// app.js — NS infrastructure  (zero dependencies, pure Vanilla JS)
NS = {
  apiRequest(url, opts)     // Axios-shaped fetch: timeout, Bearer, typed ApiError
  Auth, ApiError            // auth state + typed network errors
  showLoading(host,label)   // M2 skeleton state
  hideLoading(host)
  showError(host,msg,retry) // M2 recoverable error + Retry button
  hideError(host)
  Router, guardRoute(path)  // M3 hash router + protected-route guard
  VirtualTable.mount(...)   // M4 windowed list: visible rows only
  debounce(fn,ms)           // M5 input coalescing (cancel/flush)
  throttle(fn,ms)           // M5 scroll repaint guard
  toast / openModal / confirm  // non-blocking feedback
}
```

Pure Vanilla JS — no React, no Vue, no jQuery, no charting library. Mock data lives inside the NS layer (with a `NET` simulator that can force failures or fail the next request), rendered through DOM templates with event delegation. Because the data layer is isolated behind `apiRequest` + the NET module, a backend engineer only swaps one source to go live; the entire UI, loading and error behaviour stays intact.

- Token-driven CSS — colours, spacing and radii are `:root` variables; no magic hex values scattered in rules. Accent theming is data-attribute driven (`[data-accent="violet|emerald|amber"]`).
- Modular runtime — DOM helpers (`$` / `$$`), router, virtual list and timing utilities are each isolated and commented.
- Resilient UI — skeleton → request → render / retry is a first-class flow; the failure simulator proves the retry path without a real backend.
- Accessible & performant — visible focus, `prefers-reduced-motion` guards, and windowed rendering keep the DOM small even at 5,000 rows.

05 QA / Test Report Matrix

Every assertion below is reproducible in a browser with no backend attached. All statuses are **PASS**.

CATEGORY	TEST CASE	EXPECTED RESULT	STATUS
Runtime	Force `NET.forceFail` / `NET.failNext`	Skeleton shows, ApiError caught, Retry button renders, UI recovers on retry	PASS
Virtual list	Load 5,000 rows, scroll the table	Only visible rows (+overscan) mounted; smooth scroll; no DOM bloat	PASS
Command	Open ⌘K, search, execute an action	Palette opens, fuzzy match, navigates, closes; trigger focus restored	PASS
Auth	Logout, then deep-link a protected route	`guardRoute` intercepts; user bounced to sign-in; login restores view	PASS
Charts	Resize 320 → 1440 px	SVG scales fluidly; no horizontal overflow; data deterministic	PASS
Modal	Open / close via button / backdrop / escape	Closes cleanly; focus returns to trigger; no scroll-lock leak	PASS
Timers	Fire `toast()` 50× in a loop	Zero leaked timers; heap remains flat	PASS
Forms	Create user with invalid vs valid email/name	Invalid blocked inline; valid persists to the (mock) store	PASS
Export	Export invoices to CSV	File downloads with correct header row & rows	PASS
Browser	Chrome / Safari / Firefox / Edge (latest)	Identical render; no vendor prefixes required	PASS

06 Delivery Checklist

Sign-off summary for hand-off.

- [x] **Functional completeness**
  All in-scope views (overview, users, billing), command palette, virtual list, auth and export are clickable and produce correct state changes.
- [x] **Visual quality**
  Token-driven dark design system, hand-built inline-SVG icon sprite, glass surfaces.
- [x] **Responsive compliance**
  Collapsing sidebar; verified at 320 / 768 / 1280 px.
- [x] **Performance**
  Zero external requests; windowed rendering keeps the DOM small; opens instantly, fully offline.
- [x] **Code standards**
  Vanilla JS, NS infrastructure, commented M1–M5, no dependencies.
- [x] **Content integrity**
  Fictional brand only (Northstar); no real analytics vendors named.

NEXT STEPS (ANTI-PITFALL CLAUSE)

Front-end is delivered production-ready. The following are **explicitly out of scope** for this phase and belong to the backend / integration team — do not treat the prototype's mock success states or simulated data as live analytics:

1. Connect a real analytics API / data source; replace the mock data and `NET` simulator with live endpoints behind `apiRequest`.
2. Wire real authentication (OAuth / SSO) and server-side session management instead of the client-side mock auth.
3. Add server-side authorization & roles — scope views and actions by user role.
4. Build a real-time data pipeline — live metrics, caching and incremental updates.
5. Supply real copy, privacy policy and terms (currently placeholder / demo).